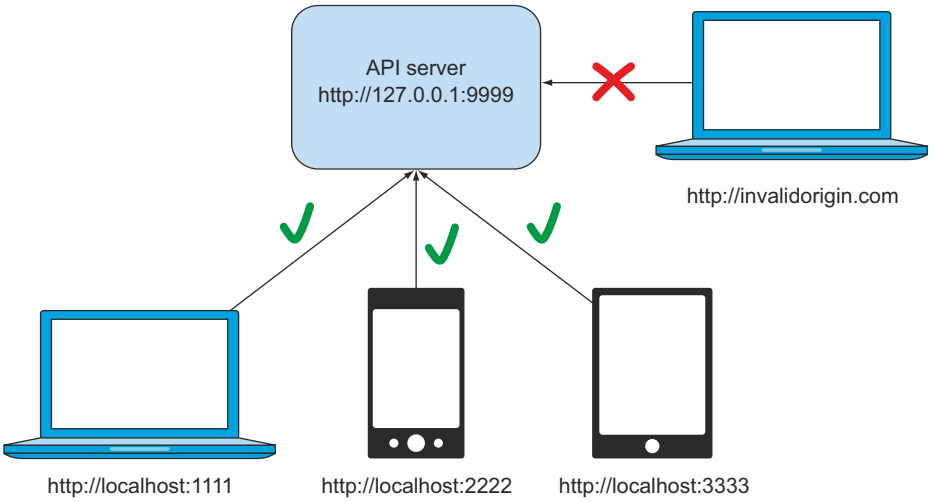


Figure 6.11 Whitelisted requests from a desktop, mobile, and tablet client

So if CORS shouldn't be used for security, how can you protect the content on your site when enabling cross-origin requests? The rest of this section explores ways to keep your site secure while still using CORS.

6.4.1 Including cookies on requests

Cookies aren't included on CORS requests by default, but they can be enabled by setting the Access-Control-Allow-Credentials header to `true`. If you're thinking about enabling cookies on your API, you should really consider whether you need cookie support. Cookies make requests harder to debug, and expose a new attack vector. So if you don't need cookies, don't enable them.

Cookies work best in situations where

- You want to authorize users within your own ecosystem of clients and servers.
- You know exactly which clients will be accessing your server.

This means that you control both the clients and the servers, and you don't accept CORS requests from external clients. The web/tablet/mobile scenario shown in figure 6.11 is a good example. In this scenario, CORS access is limited to a select few whitelisted origins. Because you're in charge of both the server and the clients, you have control over where the requests are coming from.

The following listing shows how to configure the sample code to allow cookies.

Listing 6.5 Enabling cookies on requests

```
var corsOptions = {
  allowOrigin: createWhitelistValidator(originWhitelist),
  allowCredentials: true
};
```

Sets allowCredentials option to true